JOHN THE RIPPER

Practical Hash Cracking and Password Recovery Toolkit

This module introduces **John the Ripper (JtR)**, a password-cracking tool used to recover plaintext credentials from hashes using techniques like **dictionary attacks and rule-based mutations**. It covers cracking various formats including **MD5, SHA, NTLM (NT LAN Manager), Linux /etc/shadow, ZIP/RAR archives, and SSH keys**, while emphasizing correct hash identification and tool usage. The focus is on practical application in **penetration testing and security analysis**.

Overall, the objective is to build a strong operational understanding of how password cracking works in real-world scenarios, highlighting both **security weaknesses in poor password practices** and the **practical methodologies used in penetration testing and red team engagements**.

This documentation and learning approach is strongly inspired by the structured, hands-on content provided by TryHackMe, which significantly contributed to practical understanding of John The Ripper tool concepts.

DOCUMENTED BY

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

Task1: Introduction

Overview
John the Ripper (JtR) is a widely used password-cracking tool in cybersecurity. It is designed to identify weak passwords by performing **offline attacks** on hashed credentials. Its strength lies in:

- High cracking performance (CPU/GPU optimized)
- Support for multiple hash formats
- Extensibility via custom rules and plugins

The most feature-rich version is:

- **Jumbo John** → Extended version with additional formats and tools

Learning Objectives
You will learn how to use John the Ripper for:

- Cracking Windows authentication hashes (**NTLM – NT LAN Manager**)
- Cracking Linux password hashes from /etc/shadow
- Cracking password-protected archives (**ZIP, RAR**)
- Cracking encrypted SSH private keys
- Applying advanced cracking techniques (rules, modes)

Task 2: Basic Terms (With Full Forms + Explanation)

1. What is a Hash?
A **hash** is a fixed-length output generated from input data using a **hashing algorithm**.
Examples:

Input	Algorithm	Output
polo	MD5 (Message Digest 5)	b53759f3ce692de7aff1b5779d3964da
polomints	MD5	584b6e4f4586e136bc280f27f9c64f3b

Key Properties of Hashing

1. **Deterministic**
   Same input → same hash
2. **Fixed Length Output**
   - MD5 → 128 bits (16 bytes)
   - SHA-1 (Secure Hash Algorithm 1) → 160 bits
   - SHA-256 → 256 bits
3. **Avalanche Effect**
   Small input change → large output change
4. **One-Way Function**
   Cannot reverse hash to original input

WEIRDO

2. What Makes Hashing Secure?

Hashing security is based on computational difficulty.

P vs NP Concept

Term	Full Form	Meaning
P	Polynomial Time	Problems solvable quickly
NP	Non-deterministic Polynomial Time	Solutions verifiable quickly, but hard to find

Logic

- Hashing → **Easy (P)**
- Reverse hashing → **Hard (NP)**

This makes brute-force the only practical attack method.

3. Where John the Ripper Fits

Even though hashes are not reversible:

- We can **guess inputs**
- Hash them
- Compare with target hash

This is called a:

Dictionary Attack

Process:

Password List → Hash → Compare → Match → Password Found

WEIRDO

Task 3: Setup and Environment

Required Components

1. John the Ripper (Jumbo Version)

- Full-featured version
- Includes tools like:
  - zip2john
  - rar2john
  - ssh2john

Installation Commands

Ubuntu:

sudo apt install john

Fedora:

sudo dnf install john

Verification

john

Expected output:

John the Ripper 1.9.0-jumbo-1

2. Wordlists

Definition:

A **wordlist** is a collection of potential passwords used in attacks.

RockYou Wordlist

- Source: **rockyou.com data breach**
- Contains: ~14 million passwords

Location:

/usr/share/wordlists/rockyou.txt

Extract if needed:

gzip -d rockyou.txt.gz

WEIRDO

Task 4: Cracking Basic Hashes

Core Syntax
john [options] [hash_file]

1. Automatic Mode
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Logic:

- Detect hash type automatically
- Apply suitable cracking method

2. Format-Specific Mode
john --format=raw-md5 --wordlist=rockyou.txt hash.txt

Important Flag Breakdown

Flag	Meaning
--wordlist	Specifies dictionary file
--format	Defines hash type
raw-	Indicates standard hash

Example
john --format=raw-md5 --wordlist=rockyou.txt hash.txt

Hash Identification
Use tools like:
python3 hash-id.py

Practical Results Summary

File	Hash Type	Password
hash1.txt	MD5	biscuit
hash2.txt	SHA1	kangeroo
hash3.txt	SHA256	microphone
hash4.txt	Whirlpool	colossal

WEIRDO

Task 5: Cracking Windows Hashes

NTLM (NT LAN Manager)

- Used in Windows authentication
- Stored in **SAM (Security Account Manager)**

Command

john --format=nt --wordlist=rockyou.txt ntlm.txt

Result

- Password: mushroom

Key Concept: Pass-the-Hash

Instead of cracking:

- Use hash directly to authenticate

Task 6: Cracking /etc/shadow

Files Involved

File	Purpose
/etc/passwd	User info
/etc/shadow	Password hashes

Step 1: Combine Files

unshadow passwd shadow > combined.txt

Step 2: Crack

john --wordlist=rockyou.txt --format=sha512crypt combined.txt

Result

- Root password: 1234

WEIRDO

Task 7: Single Crack Mode

Concept: Word Mangling
John modifies known data like:

- Username
- Full name

Example
Username: Markus
Generated passwords:

- Markus1
- Markus123
- MARKUS!
- M4rkus

Command
john --single --format=raw-sha256 hashes.txt

Input Format Required
username:hash

Result

- Password: Jok3r

WEIRDO

Task 8: Custom Rules

Purpose
Exploit predictable password patterns.

Common Pattern
Password → Polopassword1!

Rule Syntax
[List.Rules:RuleName]

Modifiers

Symbol	Meaning
c	Capitalize
Az	Append
A0	Prepend

Character Sets

Pattern	Meaning
[0-9]	Numbers
[A-Z]	Uppercase
[a-z]	Lowercase
[!@#]	Symbols

Example Rule
[List.Rules:PoloPassword]
cAz"[0-9][!@#]"
Command
john --wordlist=rockyou.txt --rule=PoloPassword hash.txt

WEIRDO

Task 9: Cracking ZIP Files

Step 1: Extract Hash
zip2john file.zip > hash.txt

Step 2: Crack
john --wordlist=rockyou.txt hash.txt

Result

- Password: pass123
- Flag: THM{w3ll_d0n3_h4sh_r0y4l}

Task 10: Cracking RAR Files

Step 1
rar2john file.rar > hash.txt

Step 2
john --wordlist=rockyou.txt hash.txt

Result

- Password: password
- Flag: THM{r4r_4rch1ve5_th15_t1m3}

WEIRDO

Task 11: Cracking SSH Keys

SSH Key Concept

- Private key: id_rsa
- Protected with passphrase

Step 1

ssh2john.py id_rsa > hash.txt

Step 2

john --wordlist=rockyou.txt hash.txt

Result

- Password: mango

Task 12: Final Summary

Key Tools

Tool	Purpose
john	Main cracking engine
zip2john	Extract ZIP hashes
rar2john	Extract RAR hashes
ssh2john	Extract SSH key hashes
unshadow	Combine passwd + shadow

Key Techniques

- Dictionary attack
- Word mangling
- Custom rules
- Format-specific cracking

Key Takeaway

Hash cracking is not about reversing hashes but:
Guess → Hash → Compare → Match

WEIRDO